

Artificial Intelligence & Machine Learning

– Task 5

Ensemble Learning, Model Optimization & Real-World ML Pipeline

Name: Chetan Sharma

Task: Task 5

Domain: Artificial Intelligence & Machine Learning

Dataset: Breast Cancer Dataset

Tool Used: Python / VS Code / Jupyter Notebook

Libraries: Pandas, NumPy, Matplotlib, Scikit-learn

1. Introduction

Ensemble Learning is a machine learning approach in which multiple models are combined to improve the overall performance of a prediction system. Instead of depending on a single model, ensemble methods combine the predictions of multiple learners.

In this task, different machine learning models were implemented and compared using the Breast Cancer dataset. The project focuses on **Logistic Regression, Random Forest, Gradient Boosting and Hyperparameter Tuning using GridSearchCV**.

The main objective of this task is to understand how different machine learning models perform on the same dataset and how optimization techniques can be used to improve model performance.

2. Dataset

The **Breast Cancer dataset** was used for this project. The dataset contains **569 samples and 30 features**.

The dataset was loaded using Scikit-learn:

```
from sklearn.datasets import load_breast_cancer

data = load_breast_cancer()

X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target
```

The dataset was divided into training and testing sets using an **80:20 split**.

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

The training set contained **455 samples**, while the testing set contained **114 samples**.

3. Data Preprocessing

Feature scaling was performed using `StandardScaler`. Scaling is particularly useful for models such as Logistic Regression because different features can have different numerical ranges.

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

The scaler was fitted only on the training data and then applied to the test data.

4. Models Implemented

4.1 Logistic Regression

Logistic Regression was used as the **baseline model**. It provides a reference performance against which the ensemble models can be compared.

```
model = LogisticRegression(max_iter=1000)

model.fit(X_train_scaled, y_train)

y_pred = model.predict(X_test_scaled)
```

The accuracy was then calculated using `accuracy_score`.

4.2 Random Forest Classifier

Random Forest is an ensemble learning algorithm based on multiple decision trees. It uses the predictions of multiple trees to produce the final classification.

```
rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
```

```
)  
  
rf_model.fit(X_train, y_train)  
  
rf_pred = rf_model.predict(X_test)
```

Random Forest was then evaluated using its test accuracy.

4.3 Gradient Boosting Classifier

Gradient Boosting is another ensemble technique. It builds models sequentially, where new models focus on correcting errors made by previous models.

```
gb_model = GradientBoostingClassifier(  
    n_estimators=100,  
    learning_rate=0.1,  
    random_state=42  
)  
  
gb_model.fit(X_train, y_train)  
  
gb_pred = gb_model.predict(X_test)
```

The Gradient Boosting model was also evaluated using test accuracy.

The task guide describes Gradient Boosting as a sequential learning method based on error correction.

5. Hyperparameter Tuning

Hyperparameter optimization was performed using **GridSearchCV**. The purpose of GridSearchCV is to test different combinations of hyperparameters and identify the combination that provides the best cross-validation performance.

The following parameters were tuned:

- `n_estimators`
- `max_depth`
- `min_samples_split`

```
param_grid = {  
    'n_estimators': [50, 100, 200],  
    'max_depth': [None, 5, 10],  
    'min_samples_split': [2, 5]  
}  
  
grid_search = GridSearchCV(  
    estimator=RandomForestClassifier(random_state=42),  
    param_grid=param_grid,  
    cv=5,
```

```
        scoring='accuracy',
        n_jobs=-1
    )

    grid_search.fit(X_train, y_train)
```

The best parameters were obtained using:

```
print(grid_search.best_params_)
print(grid_search.best_score_)
```

The optimized Random Forest model was then used for final prediction.

The task guide specifically emphasizes **GridSearchCV**, **parameter tuning** and **cross-validation** as important parts of model optimization.

6. Model Comparison

The following models were compared:

1. Logistic Regression
2. Random Forest
3. Gradient Boosting
4. Optimized Random Forest

The accuracy of each model was calculated and presented in a comparison table and graph.

Model Comparison

Model	Accuracy
Logistic Regression	[0.9824561403508771]
Random Forest	[0.956140350877193]
Gradient Boosting	[0.956140350877193]
Optimized Random Forest	[0.956140350877193]

The models were compared based on their accuracy, and the model with the strongest performance was selected for final evaluation.

7. Final Model Evaluation

The optimized Random Forest model was selected for final evaluation.

The following metrics were obtained using the classification report:

- Precision

- Recall
- F1-score
- Accuracy

```
print(classification_report(y_test, best_rf_pred))
```

Final accuracy was calculated using:

```
final_accuracy = accuracy_score(y_test, best_rf_pred)

print("Final Model: Optimized Random Forest")
print("Final Accuracy:", final_accuracy)
```

Final Result

Final Model: Optimized Random Forest

Final Accuracy: [0.956140350877193]

Precision: [0.95,0.96,0.96,0.96]

Recall: [0.93,0.97,0.95,0.96]

F1-score: [0.94,0.97,0.96,0.95,0.96s]

8. Conclusion

This project provided practical experience with **Ensemble Learning and Model Optimization**. Logistic Regression was first implemented as a baseline model. Random Forest and Gradient Boosting were then implemented as ensemble learning approaches.

GridSearchCV was used to optimize the Random Forest hyperparameters using cross-validation. The performance of all models was compared using their test accuracy.

The project demonstrated that model selection and hyperparameter tuning are important steps in developing an effective machine learning pipeline. The final optimized model was evaluated using accuracy, precision, recall and F1-score.

Overall, this task helped in understanding **ensemble methods, Random Forest, Gradient Boosting, hyperparameter tuning, cross-validation, model comparison and end-to-end machine learning workflow**. These topics are also the main learning objectives identified in the provided Task 5 guide.
